

2026 CCF 开源创新大赛

「基于 openvela 的智能手环应用开发」参赛作品

openvela Smart Band Demo

项目实施报告



项目形态： openvela/NuttX 原生智能手环应用

核心技术： C、LVGL、uORB、libuv、goldfish 模拟器

编写日期： 2026 年 7 月

摘要

本项目面向 2026 CCF 开源创新大赛“基于 openvela 的智能手环应用开发”赛题，完成了一套可在 openvela/NuttX 环境中运行的原生智能手环应用。项目采用 C 语言与 LVGL 构建小屏图形界面，以 goldfish 模拟器作为主要开发和验证平台，实现表盘、心率、计步和应用中心四个主页面，并支持左右滑动循环切换、应用进入与返回、状态刷新和按钮交互。除基础健康信息展示外，应用中心还集成天气、计算器、倒计时、2048、秒表、扫雷、俄罗斯方块和电子木鱼八个小应用，覆盖数据展示、常用工具、益智游戏和轻量互动等多种使用场景，使项目不再局限于单一表盘展示，而是具备较完整的小型手环应用形态。

在系统设计上，项目将界面显示、状态管理、传感器访问和小应用逻辑进行分层处理。页面层主要负责 LVGL 控件创建、布局和交互事件；统一状态模型负责保存时间、心率、步数、温湿度、电池容量、充电状态以及各类传感器有效标志；独立的传感器桥接层负责访问 goldfish 与 uORB 提供的设备节点，并将不同设备的数据转换为页面可直接使用的状态字段。当前已接入心率、计步、三轴加速度、环境温度、普通温度、湿度和电池设备。采集过程采用非阻塞方式，定时读取队列中的最新完整样本，避免传感器访问阻塞 LVGL 与 libuv 的界面循环。传感器数据更新后先写入统一模型，再由页面集中刷新，从而避免设备代码直接操作界面控件，降低模块之间的耦合。

针对模拟器和实际开发板中可能出现的设备节点缺失、样本暂未到达、接口不支持或读取失败等情况，项目设置了多种降级策略。计步数据优先使用独立计步器，在计步器不可用时根据加速度变化推导步数；环境温度节点打开失败时自动尝试普通温度节点；非阻塞读取暂时没有新样本时保留上一次有效值或模型默认值，避免页面数值频繁归零或出现空白；电池接口读取失败时仅跳过本轮更新，不影响其他页面继续运行。项目还对心率、步数目标、温湿度、电量、倒计时、计算器输入和游戏棋盘范围等关键数据设置了边界检查，并对显示初始化失败、界面创建失败和资源释放流程进行了处理，以提高程序在异常场景下的稳定性。

仓库同时提供模型自动测试、模拟器传感器联动脚本、运行说明和一键复现脚本，用于验证页面切换、步数进度、时间处理、传感器数据传递、应用交互和异常降级等功能。通过上述实现，本项目完成了从前端界面原型到 openvela 原生应用的迁移，并形成了较为清晰的界面层、模型层、桥接层和设备层结构，为后续移植到真实手环硬件、增加蓝牙同步、消息通知、数据持久化和更精确的健康算法提供了可扩展基础。

关键词：openvela；智能手环；LVGL；NuttX；uORB；传感器；嵌入式应用

目录

摘要	I
第一章 项目背景与参赛定位	1
1.1 项目背景	1
1.2 实现要点	2
第二章 需求分析与总体设计	3
2.1 功能需求	3
2.2 非功能需求	4
2.3 系统分层架构	7
2.4 主循环与数据流	7
第三章 工程结构与构建集成	8
3.1 目录组织	8
3.2 Kconfig 配置	8
3.3 双构建入口	9
第四章 核心模型与页面交互实现	10
4.1 统一状态模型	10
4.2 时间与时区处理	10
4.3 四页循环切换	10
4.4 步数目标与进度	12
4.5 小屏布局策略	12
第五章 传感器桥接与数据闭环	13
5.1 桥接层的作用	13
5.2 从模拟器到 LVGL 的完整链路	13
5.3 桥接生命周期	14
5.4 每秒更新顺序与覆盖规则	15

5.5	桥接输入与页面落点	17
5.6	设备节点	18
5.7	非阻塞读取与最新样本	18
5.8	计步器优先与加速度回退	19
5.9	温湿度与电池状态	20
第六章 应用中心与关键算法		21
6.1	统一应用接口	21
6.2	八个应用功能	21
6.3	计算器状态机与除零处理	23
6.4	2048 的移动与合并	23
6.5	扫雷的递归展开	25
6.6	倒计时边界	26
6.7	电子木鱼反馈设计	26
第七章 编译、运行与一键复现		27
7.1	环境要求	27
7.2	手动接入源码	27
7.3	配置项	27
7.4	编译过程	28
7.5	启动模拟器	28
7.6	Skill 一键复现	29
第八章 异常处理与可靠性设计		31
8.1	显示和生命周期错误	31
8.2	传感器异常降级	31
8.3	输入和数值边界	32
8.4	复现脚本错误处理	32
第九章 测试与验证		33
9.1	模型自动测试	33
9.2	传感器联动测试	33
9.3	人工验收流程	33

9.4 验证边界说明	34
第十章 开源合规与可维护性	35
10.1 开源文件	35
10.2 扩展新应用	35
第十一章 总结与展望	36
附录 A 快速复现命令清单	38

第一章 项目背景与参赛定位

1.1 项目背景

智能手环的屏幕和内存都比较有限，但仍要同时处理时间、健康数据、设备状态和轻量应用。因此，界面实现之外还要解决设备节点读取、主循环响应、触摸误操作、字体资源和板级差异等问题。

项目最初只有浏览器原型。为了接入 openvela 的传感器与电池接口，后续将主要功能改写为 C + LVGL 原生应用，并在 NuttX/goldfish 环境中运行。浏览器版本继续保留，用来快速查看布局；编译、传感器和设备状态的验证则以原生程序为准。

表 1.1: 赛题要求与项目实施映射

赛题关注点	项目实施	验证方式
表盘与时间显示	当前时间、日期、电量、充电状态、睡眠、心率、压力和温度摘要	启动后观察表盘；时间按 UTC+8 修正
健康数据展示	心率和步数优先读取模拟器传感器；步数支持加速度回退	调节模拟器传感器并观察 UI 刷新
基础 UI 交互	四个主页面左右滑动循环切换；应用图标可点击进入和返回	手势、圆点导航和按钮操作
应用完整性	八个独立小应用，包含工具类、数据类和游戏类功能	逐项进入并执行核心操作
可运行与可复现	Kconfig、CMake、legacy Make、运行手册和一键复现 Skill	执行复现脚本并启动模拟器
基本异常处理	显示失败退出、传感器缺失降级、输入与数值边界保护	日志、模型测试和缺失设备场景

1.2 实现要点

原生运行与界面实现

项目不是通过 WebView 加载网页，也不是仅提供浏览器原型，而是以 NuttX 应用命令的形式接入 openvela。smart_band_main.c 负责完成 LVGL 初始化、NuttX 显示适配、界面创建以及 libuv 事件循环启动。程序编译完成后，可以在模拟器的 NuttShell 中直接输入 smart_band 启动。浏览器 Demo 只用于快速查看页面布局，真正的传感器读取、状态更新和交互逻辑均在 openvela 原生程序中执行。

设备桥接与数据更新

所有传感器和电池访问统一集中在 sensor_bridge.c 中，页面代码不直接调用 open()、read() 或 ioctl()。桥接层负责读取心率、计步、加速度、温度、湿度和电池设备节点，并将原始数据转换为 smart_band_state_t 中的统一字段。界面定时器每秒先更新基础模型，再读取传感器数据，随后更新当前应用，最后统一刷新页面。只有读取到完整有效样本时才覆盖当前状态；若本轮暂时没有新数据，则保留上一次有效值，避免页面出现数值归零或闪烁。

模块拆分与异常降级

项目将状态模型、主页面、传感器桥接和八个小应用分别放在独立文件中维护。watch_model.c 负责时间、页面编号、步数目标和默认数据，app_lvgl.c 负责四个主页面的布局与交互，smart_band_apps.c 负责应用注册和分发，各小应用则分别位于 apps/ 目录。传感器统一以非阻塞方式打开，单个设备缺失不会导致应用退出。计步器不可用时使用加速度变化估算步数，环境温度节点不存在时尝试普通温度节点，电池读取失败时只跳过本轮更新。项目同时提供 CMake 和 legacy Make 构建入口、模型测试、传感器联动脚本、运行说明及开源声明，便于重新构建和验证。

第二章 需求分析与总体设计

2.1 功能需求

项目需要覆盖设备状态、健康数据、页面交互和小应用四类功能，并保证各项功能能够在 openvela 模拟器中实际操作和验证，而不是只完成静态页面展示。

表盘与设备状态显示

表盘需要实时显示当前时间、日期、电池剩余容量和充电状态。时间信息由统一状态模型定时更新，并根据模拟器环境完成时区处理；电池容量通过 goldfish 电池设备接口读取，以百分比和图形进度两种方式展示。检测到充电状态时，界面同步显示充电标识，使用户无需进入其他页面即可了解设备的基本运行状态。

健康与环境数据展示

系统需要显示心率、累计步数、环境温度和相对湿度等数据。心率优先从 uORB 心率设备节点读取，步数优先从 uORB 计步设备节点获取，温度和湿度分别通过相应的 uORB 设备节点采集。读取到的新数据写入 smart_band_state_t，再由表盘摘要、心率页面、计步页面和天气应用分别显示，避免多个页面直接访问设备文件。

四个主页面循环切换

主界面由表盘、心率、计步和应用中心四个页面组成，用户可以通过左右滑动进行双向循环切换。页面切换采用取模方式处理，因此从应用中心继续向后滑动可以返回表盘，从表盘向前滑动也可以进入应用中心。手势识别需要同时比较水平位移和垂直位移，只有达到滑动阈值时才执行翻页，避免按钮点击被误判为页面手势。

步数目标和完成进度

计步页面需要支持用户调整每日步数目标，单次调整单位为 1000 步，目标范围限制在 1000 至 50000 步之间。页面根据当前步数和目标步数实时计算完成比例、剩余

步数和进度条数值。完成比例限制在 0 至 100 之间，防止步数超过目标后造成进度条越界；目标调整后，相关数值需要立即同步更新。

八个小应用的完整交互

应用中心需要提供天气、计算器、倒计时、2048、秒表、扫雷、俄罗斯方块和电子木鱼八个应用。每个应用必须能够从应用中心进入，并能够通过统一返回按钮回到应用列表。应用内部不能只显示固定图片或静态文字，而应具备可验证的操作，例如计算器执行四则运算、倒计时开始和暂停、2048 进行滑动合并、扫雷切换难度、俄罗斯方块移动与下落、电子木鱼累计点击次数等。

传感器异常和设备缺失处理

当某个传感器设备不存在、暂时没有新样本或读取失败时，应用仍需保持正常运行。传感器以非阻塞方式打开和读取，避免设备访问卡住 LVGL/libuv 主循环。计步器不可用时，程序需要根据加速度变化估算步数；环境温度节点不可用时，需要继续尝试普通温度节点；电池读取失败时只跳过本轮更新，不影响心率、计步和其他页面。所有失败情况应通过日志或页面数据来源状态体现。

多种运行与验证入口

项目需要同时提供浏览器预览、openvela 原生编译运行和一键复现三种验证方式。浏览器 Demo 用于快速检查页面布局和基础交互；openvela 原生程序用于验证 LVGL 运行、设备节点读取和传感器数据链路；复现脚本负责同步源码、写入构建配置、执行编译并给出模拟器启动命令。三种入口用途不同，其中传感器功能必须以 openvela 模拟器中的原生程序为准。

2.2 非功能需求

小屏可读性

界面需要适应智能手环的小尺寸屏幕。时间、心率、步数和倒计时等主要数据使用较大字号，标题、单位和来源说明使用较小字号，形成明确的视觉层级。卡片、按钮、棋盘和进度条采用固定尺寸，避免数值长度变化时发生明显跳动或重叠。应用中心图标和按钮应保留足够的触摸区域，减少小屏条件下的误操作。

模块低耦合

状态模型、传感器读取、页面绘制和小应用逻辑需要分别维护。`watch_model.c` 负责时间、页面编号、步数目标和基础状态；`sensor_bridge.c` 负责设备节点访问和数据转换；`app_lvgl.c` 负责主页面布局、手势和刷新；`smart_band_apps.c` 负责应用注册与分发。页面层只读取统一模型，不直接保存传感器文件描述符，也不直接执行设备读写操作。

界面响应性

传感器设备统一使用非阻塞只读方式打开，读取函数只处理当前队列中已经到达的完整样本。定时更新过程不能长时间等待设备数据，也不能在 LVGL 事件回调中执行阻塞操作。每秒更新时依次处理模型、传感器、活动应用和页面渲染，使界面滑动、按钮点击和动画不会因为传感器暂时无数据而失去响应。

运行稳定性

程序需要处理显示初始化失败、UI 创建失败、传感器设备缺失、无效文件描述符和数值越界等情况。所有文件描述符初始值设为 -1，读取和关闭前检查有效性；UI 创建失败时释放已经初始化的资源；传感器没有新样本时保留上一次有效值或模型默认值，避免页面出现空白、频繁归零或程序崩溃。

构建兼容性

项目需要同时保留 CMake 和 NuttX legacy Make 两种构建入口。`CMakeLists.txt` 通过 `nutt_x_add_application()` 注册 `smart_band` 命令，`Makefile` 和 `Make.defs` 用于兼容仍采用传统构建方式的源码树。两种构建方式应引用相同的源文件列表，避免不同构建入口生成的程序功能不一致。

可维护性与扩展性

每个小应用使用独立的 C 源文件实现，并通过统一的 `build`、`update` 和 `tick` 接口接入应用中心。新增应用时，主要完成应用文件、应用 ID、图标资源和构建列表注册，不需要修改传感器底层或重写整个页面框架。设备路径变化时，应优先在桥接层增加路径选择或板级配置，而不是在多个页面中分别处理。

资源占用可控

项目面向资源受限的嵌入式设备，运行时应避免频繁创建和销毁页面对象。四个主页面在初始化时建立，切换时主要调整可见状态；图标资源转换为 LVGL C 数组并随程序编译，不依赖运行时文件系统加载 PNG。步数比例、页面编号和大部分状态计算采用整数运算，减少不必要的计算和存储开销。

验证与复现性

仓库需要提供明确的构建配置、运行说明、自动测试和传感器联动脚本。模型测试用于检查时间格式、页面循环、步数目标和进度计算；传感器脚本用于向模拟器写入心率、温度、湿度、加速度和电池状态；人工验收流程用于检查页面滑动、应用进入、返回和按钮操作。复现过程应尽量减少手工修改，并在缺少环境或构建失败时给出明确提示。

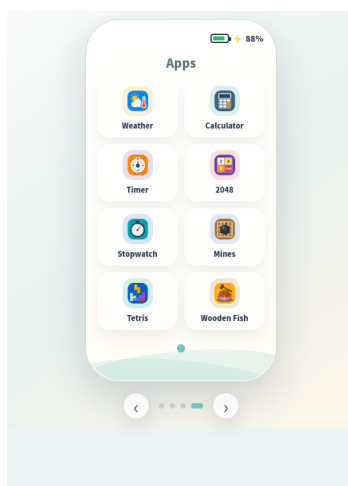


图 2.1: 应用中心的两列入口布局

2.3 系统分层架构

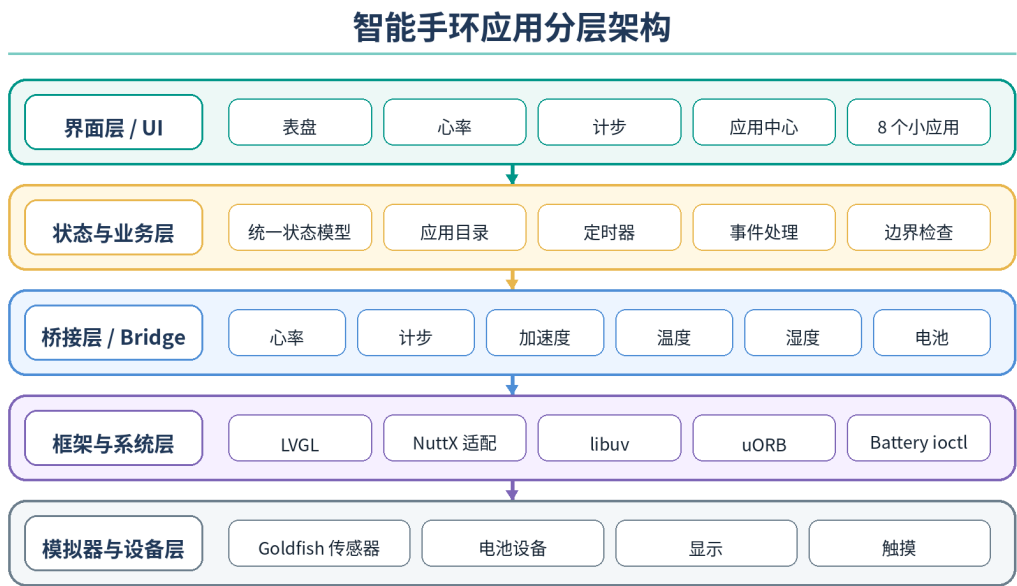


图 2.2: 智能手环应用分层架构

设备读取集中在 `sensor_bridge.c`, 页面状态由 `watch_model.c` 保存, `app_lvgl.c` 只负责创建控件和刷新显示。小应用通过统一 `host` 接口使用公共能力, 不直接接触设备文件描述符。

2.4 主循环与数据流

程序入口完成 LVGL 初始化、NuttX 显示适配和 UI 创建, 然后把 LVGL 交给 libuv 事件循环。UI 内部定时器每秒依次更新时间、模型、传感器和活动应用, 最后统一渲染当前页面。

模拟器设备节点 → `sensor bridge` → `smart_band_state_t` → LVGL 控件 → 屏幕

第三章 工程结构与构建集成

3.1 目录组织

原生应用位于 `openvela_app/smart_band`，接入 `openvela` 后目标位置为：

`packages/demos/smart_band_basic`

核心结构如下。

Listing 3.1: 原生应用目录结构

```
1 smart_band/
2 |-- smart_band_main.c      # program entry and LVGL/libuv lifecycle
3 |-- app_lvgl.c             # four main pages and shared UI helpers
4 |-- watch_model.c          # time, page state and fallback model
5 |-- sensor_bridge.c        # uORB sensors and battery bridge
6 |-- smart_band_apps.c      # application catalog and dispatch
7 |-- apps/                  # one source file per mini application
8 |-- assets/generated_icons/ # original PNG icon assets
9 |-- icon_assets.c          # icons converted to LVGL C data
10 |-- include/               # public headers
11 |-- Kconfig
12 |-- CMakeLists.txt
13 |-- Makefile
14 `-- Make.defs
```

小应用按独立源文件组织，状态、算法和事件处理可以分别定位，主 UI 文件只保留页面框架和公共能力。

3.2 Kconfig 配置

应用配置项依赖 LVGL NuttX 适配、libuv UI 循环、传感器框架和 uORB。默认任务优先级为 100，栈大小为 32768 字节。

Listing 3.2: 关键 Kconfig 约束

```
1 config LVX_USE_DEMO_SMART_BAND_BASIC
2     bool "Smart band basic LVGL demo"
3     depends on GRAPHICS_LVGL && LV_USE_NUTTX && LV_USE_NUTTX_LIBUV
4     depends on SENSORS && UORB
```

```
5   default n
6
7   config LVX_DEMO_SMART_BAND_BASIC_PRIORITY
8     int "Smart band basic task priority"
9     default 100
10
11  config LVX_DEMO_SMART_BAND_BASIC_STACKSIZE
12    int "Smart band basic stack size"
13    default 32768
```

3.3 双构建入口

CMakeLists.txt 使用 `nuttx_add_application()` 注册 `smart_band` 命令；Makefile 与 Make.defs 则兼容 legacy NuttX 构建。两条路径共享同一组 C 源文件，避免行为分叉。

Listing 3.3: CMake 中的应用注册（节选）

```
1 set(PROGNAME smart_band)
2 set(MAINSRC smart_band_main.c)
3 set(CSRCS
4   watch_model.c sensor_bridge.c app_lvgl.c icon_assets.c
5   smart_band_apps.c apps/weather_app.c apps/calculator_app.c
6   apps/timer_app.c apps/music_app.c apps/stopwatch_app.c
7   apps/mines_app.c apps/tetris_app.c apps/wooden_fish_app.c)
8
9 nuttx_add_application(
10   NAME ${PROGNAME}
11   PRIORITY ${PRIORITY}
12   STACKSIZE ${STACKSIZE}
13   INCLUDE_DIRECTORIES ${INCDIR}
14   SRCS ${MAINSRC} ${CSRCS})
```

第四章 核心模型与页面交互实现

4.1 统一状态模型

`smart_band_state_t` 保存页面编号、时间文本、心率、步数目标、电量、温湿度以及各传感器是否有效等状态。UI 每秒从模型读取结果，而不是让传感器回调直接修改多个控件。模型初始化时写入可显示的默认值，因此模拟器尚未产生第一帧数据时页面不会出现未初始化内容。

4.2 时间与时区处理

NuttX 模拟器环境中的时间曾出现比本地时间晚 8 小时的问题。项目在 NuttX 分支中对时间戳增加 UTC+8 偏移, 再通过 `gmtime_r()` 生成展示时间; POSIX 浏览器/测试环境继续使用系统本地时间。

Listing 4.1: NuttX 与 POSIX 的时间转换分支

```
1 #define SMART_BAND_UTC_OFFSET_SECONDS (8 * 60 * 60)
2
3 #if defined(__NuttX__)
4     now += SMART_BAND_UTC_OFFSET_SECONDS;
5     tm_result = gmtime_r(&now, display_time);
6 #elif defined(_POSIX_VERSION)
7     tm_result = localtime_r(&now, display_time);
8 #else
9     tm_result = localtime(&now);
10 #endif
```

当时间转换失败时, 模型显示 `--:--` 与 `----/--/--`, 并将 `time_valid` 设为 `false`, 避免展示未初始化的日期数据。

4.3 四页循环切换

四个主页面按表盘、心率、计步、应用中心排序。页面切换采用模运算, 因此最后一页继续后翻可以回到第一页, 第一页向前翻也可进入最后一页。

Listing 4.2: 双向循环页面判断

```
1 void smart_band_next_page(smart_band_state_t *state)
2 {
3     state->page = (smart_band_page_t)
4         ((state->page + 1) % SMART_BAND_PAGE_COUNT);
5 }
6
7 void smart_band_prev_page(smart_band_state_t *state)
8 {
9     state->page = (smart_band_page_t)
10         ((state->page + SMART_BAND_PAGE_COUNT - 1) %
11             SMART_BAND_PAGE_COUNT);
12 }
```

LVGL 手势处理保存按下坐标，在释放时比较水平和垂直位移。只有水平距离超过阈值且大于垂直距离时才触发翻页，降低点击按钮时被误判为滑动的概率。

Listing 4.3: 手势方向与阈值判断

```
1 dx = point.x - g_ui.press_point.x;
2 dy = point.y - g_ui.press_point.y;
3
4 if (abs_coord(dx) < threshold || abs_coord(dx) <= abs_coord(dy))
5 {
6     return;
7 }
8
9 if (dx < 0)
10 {
11     next_page();
12 }
13 else
14 {
15     prev_page();
16 }
```



图 4.1: 健康数据页面及其主要信息层级

4.4 步数目标与进度

目标步数默认值为 8000，可按 1000 步调节，并限制在 1000 至 50000 之间。完成比例采用整数运算并限制到 0–100，适合嵌入式环境且不会让进度条越界。

Listing 4.4: 目标步数和进度边界

```
1 state->step_goal = clamp_int(state->step_goal + delta,  
2                             SMART_BAND_STEP_GOAL_MIN,  
3                             SMART_BAND_STEP_GOAL_MAX);  
4  
5 return clamp_int((state->steps * 100) / state->step_goal, 0, 100);
```

4.5 小屏布局策略

UI 使用统一缩放函数把设计坐标映射到实际屏幕尺寸。主要数据使用较大字号，辅助标签使用 12–16 号字体；卡片、按钮和棋盘具有稳定尺寸，避免数值变化引起布局跳动。电池区域同时显示百分比、外框、动态填充和充电闪电图标，使状态无需依赖单一文本即可识别。

图标原始 PNG 被转换到 `icon_assets.c`，随应用编译进镜像。运行时不依赖可写文件系统或外部图片路径，更符合嵌入式应用部署方式。

第五章 传感器桥接与数据闭环

5.1 桥接层的作用

浏览器原型只需要准备一组展示数据，原生程序还要处理设备节点是否存在、样本是否到达、读取是否会阻塞以及数据何时刷新。项目把这些细节放在 `sensor_bridge.c` 与 `sensor_bridge.h` 中，上层页面只读取统一的 `smart_band_state_t`，不直接调用设备接口。

桥接层承担四项职责：

1. 在 UI 创建时打开 uORB 传感器和 goldfish 电池设备，并保存长期有效的文件描述符；
2. 每秒以非阻塞方式取出最新样本，把不同设备结构转换为页面使用的整数和布尔状态；
3. 按“专用传感器优先、替代传感器回退、模型数据兜底”的顺序决定最终数据源；
4. 在 UI 创建失败或退出时统一关闭设备，避免页面代码各自管理设备生命周期。

5.2 从模拟器到 LVGL 的完整链路

传感器数值从模拟器控制台到页面显示，要依次经过驱动、设备节点、桥接层和状态模型。

传感器数据从模拟器进入界面的路径

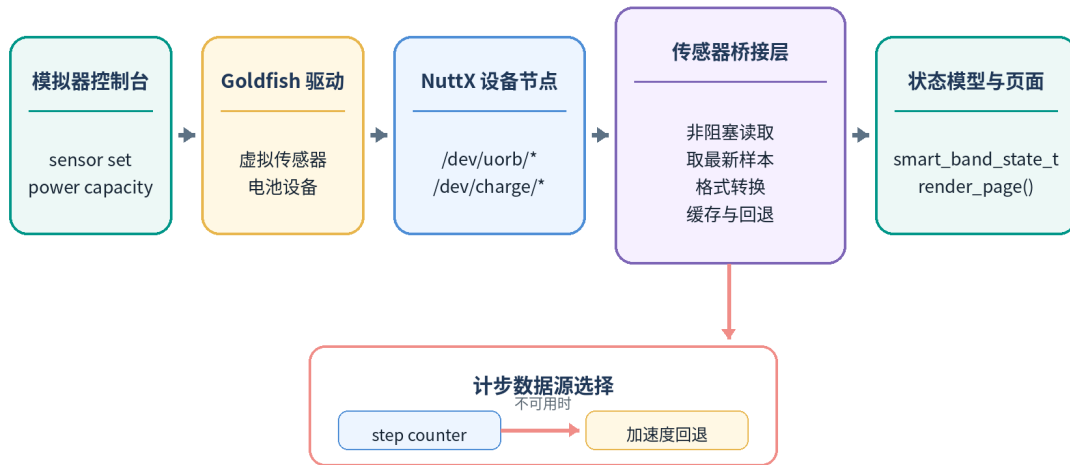


图 5.1: 模拟器、设备节点、桥接层和页面之间的数据路径

以心率为例：模拟器控制台写入 `heart-rate` 后，goldfish 驱动把样本发布到 `/dev/uorb/sensor_hrater0`。桥接层读取样本，将 bpm 转成页面使用的整数，写入 `state->heart_rate`，并更新 `heart_sensor_active`。下一次 `render_page()` 调用时，表盘摘要和心率页会一起刷新。温度、湿度、步数和电池按同样的方向进入状态模型。

5.3 桥接生命周期

桥接对象和 UI 一同创建和释放。程序先初始化状态模型和设备文件描述符，再创建 UI 树与 1 秒定时器；如果创建过程失败，或者应用退出，则在同一释放路径中关闭所有文件描述符。

Listing 5.1: UI 与传感器 bridge 的生命周期绑定

```

1 smart_band_state_init(&g_ui.model, time(NULL));
2 smart_band_sensor_bridge_init(&g_ui.sensors);
3
4 if (create_ui_tree(root) != 0)
5 {
6     smart_band_sensor_bridge_deinit(&g_ui.sensors);
7     return -1;
8 }
9
10 g_ui.timer = lv_timer_create(timer_cb, 1000, NULL);
  
```

```

11 smart_band_sensor_bridge_update(&g_ui.sensors, &g_ui.model);
12 render_page();

```

桥接层与 UI 的生命周期

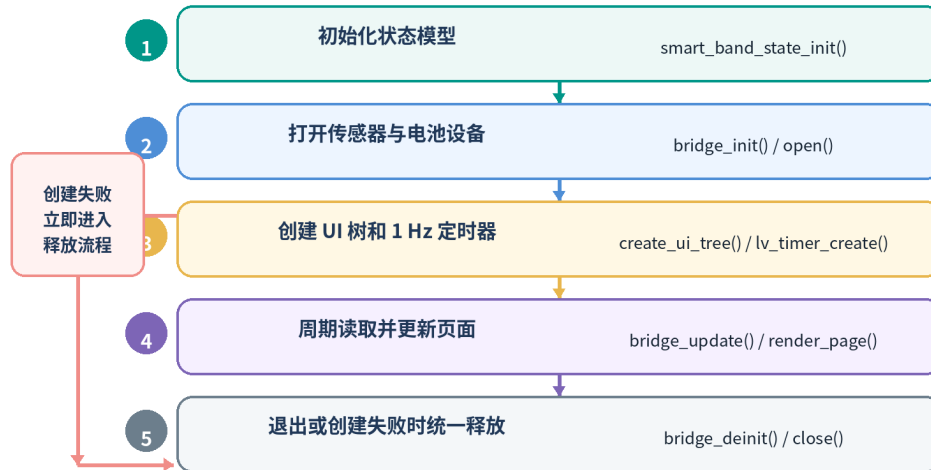


图 5.2: 桥接对象的创建、周期更新与释放路径

`smart_band_sensor_bridge_t` 不保存 LVGL 控件指针，只保存设备 fd、上次温湿度、上次加速度能量及“是否已有有效样本”等桥接状态。因此设备读取和控件更新之间始终隔着状态模型。

5.4 每秒更新顺序与覆盖规则

页面使用 1 秒定时器统一驱动更新。每次回调先调用 `smart_band_state_tick()` 更新时间并生成可用的模型值，再调用 `bridge`。`bridge` 若读到真实样本便覆盖对应字段；若某设备本轮没有样本，模型值或上次有效值继续保留。最后更新活动应用并统一渲染。

Listing 5.2: 每秒一次的模型、传感器和 UI 更新时序

```

1 static void timer_cb(lv_timer_t *timer)
2 {
3     smart_band_app_host_t host;
4
5     (void)timer;
6     smart_band_state_tick(&g_ui.model, time(NULL));
7     smart_band_sensor_bridge_update(&g_ui.sensors, &g_ui.model);
8     host = make_app_host();

```

```
9 smart_band_apps_tick(g_ui.active_app, &host);  
10 render_page();  
11 }
```



图 5.3: 每秒刷新时的执行顺序与无数据回退规则

更新结果分为三种情况：

- **读到真实样本**：bridge 覆盖模型字段，并用 active 标志告诉 UI 当前值来自传感器；
- **暂时没有新样本**：不写入无效数据，温湿度等连续量保留上次有效值；
- **设备不可用**：保持模型兜底值，只降级对应指标，不中断其他传感器和页面。

表 5.1: bridge 数据覆盖与降级语义

场景	bridge 行为	页面结果
设备存在且有新样本	转换样本、覆盖模型字段、设置 active 标志	显示来自传感器的值和 Sensor 来源
设备存在但暂时无样本	不写入未准备的数据；温湿度可保留最后有效值	页面连续，不闪回 0 或空白
设备不存在或打开失败	fd 保持 -1，读取函数立即返回	保留模型兜底值，其他传感器继续工作
专用计步器不存在	尝试从加速度能量变化推导步数	仍可演示运动到步数的桥接链路
环境温度节点不存在	初始化时尝试普通温度节点	天气页面继续获得温度来源

5.5 桥接输入与页面落点

表 5.2: 传感器字段经过 bridge 后的 UI 落点

输入数据	状态字段	最终显示位置
心率 bpm	heart_rate	表盘 Heart Rate 卡片、心率详情主数值
累计步数	steps	计步详情、目标进度、剩余步数
加速度 xyz	推导后写入 steps	无 step counter 时驱动计步详情变化
环境温度	temperature_c	表盘 Weather 卡片、天气应用温度
相对湿度	humidity_percent	天气应用 Humidity 指标
电池容量	battery_percent	所有主页面与应用页的数字和电池填充
电池状态	battery_charging	电池旁的充电闪电标识

5.6 设备节点

表 5.3: 传感器和设备节点映射

数据	设备节点	用途与策略
心率	/dev/uorb/sensor_hrate0	读取 bpm, 更新心率页和表盘摘要
计步器	/dev/uorb/sensor_step_counter0	优先使用硬件/模拟器累计步数
加速度	/dev/uorb/sensor_accel0	计步器不可用时根据运动变化推导步数
环境温度	/dev/uorb/sensor_ambient_temp0	天气温度主数据源
温度回退	/dev/uorb/sensor_temp0	环境温度节点不可用时尝试
湿度	/dev/uorb/sensor_humi0	天气应用 Humidity 数值
电池	/dev/charge/goldfish_battery	容量百分比和是否充电

5.7 非阻塞读取与最新样本

所有传感器以 `O_RDONLY|O_NONBLOCK` 打开, 并尝试设置 1 秒采样间隔。UI 定时器调用 `read_latest()` 时会读完当前队列, 只保留最后一个完整样本, 既不阻塞界面, 也避免积压数据导致页面显示过时值。

Listing 5.3: 非阻塞传感器与最新样本读取

```
1 static int open_sensor(const char *path)
2 {
3     int fd = open(path, O_RDONLY | O_NONBLOCK);
4     if (fd < 0)
5     {
6         return -1;
7     }
8
9     ioctl(fd, SNIOC_SET_INTERVAL, SMART_BAND_SENSOR_INTERVAL_US);
10    return fd;
11 }
```

```
12
13 static bool read_latest(int fd, void *sample, size_t sample_size)
14 {
15     bool updated = false;
16
17     if (fd < 0)
18     {
19         return false;
20     }
21
22     while (read(fd, sample, sample_size) == (ssize_t)sample_size)
23     {
24         updated = true;
25     }
26
27     return updated;
28 }
```

5.8 计步器优先与加速度回退

模拟器或开发板不一定提供独立 **step counter**。更新流程先读计步器，仅当本轮没有得到有效值时才调用加速度推导逻辑。这一判断使硬件计步结果具有最高优先级，同时保证缺少计步器的环境仍能演示步数变化。

Listing 5.4: 计步数据源选择

```
1 update_steps_from_counter(bridge, state);
2 if (!state->step_sensor_active)
3 {
4     update_steps_from_accel(bridge, state);
5 }
```

加速度回退将三个轴绝对值之和记为运动能量，与上一帧能量比较。当变化量超过 1.2 时计为一步，超过 4.0 时计为两步。该方法只用于模拟器联调，不等同于真实硬件上的计步算法；它的作用是确认加速度样本能够经过桥接层影响页面步数。

Listing 5.5: 加速度变化推导步数

```
1 energy = abs_float(sample.x) + abs_float(sample.y) +
2         abs_float(sample.z);
3 if (bridge->have_last_accel)
4 {
5     delta = abs_float(energy - bridge->last_accel_energy);
6     if (delta > 1.2f)
7     {
8         bridge->derived_steps += delta > 4.0f ? 2 : 1;
```

```
9     state->steps = clamp_int(state->steps + bridge->derived_steps,  
10                             0, 99999);  
11     bridge->derived_steps = 0;  
12 }  
13 }
```

5.9 温湿度与电池状态

温度读取保留最后一次有效值，避免非阻塞读取暂时返回空时温度跳回默认值。环境温度节点打开失败时自动尝试普通温度节点。湿度限制在 0–100%，并直接供天气应用使用。

电池通过 `BATIOC_CAPACITY` 和 `BATIOC_STATE` 两个 `ioctl` 分别读取容量和状态。容量驱动电池图形的填充宽度，`BATTERY_CHARGING` 则控制闪电标识，数字百分比始终保留。

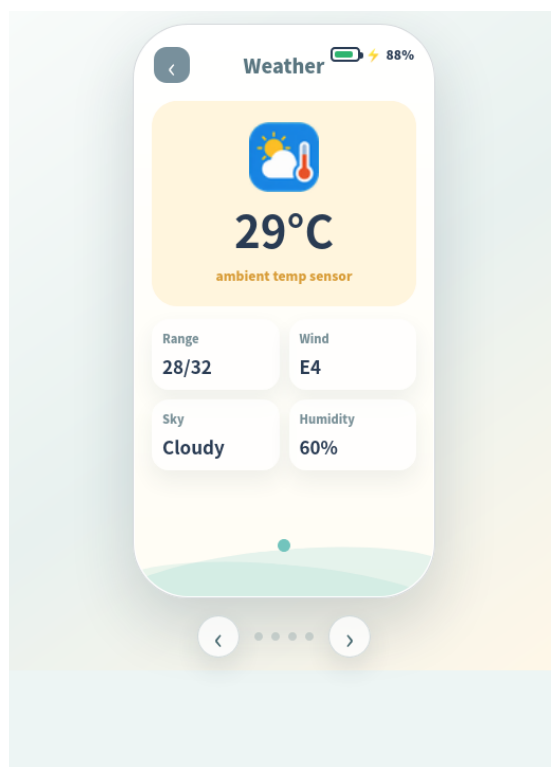


图 5.4: 天气应用读取温度与湿度后的页面效果

第六章 应用中心与关键算法

6.1 统一应用接口

应用目录在 `smart_band_apps.c` 中维护。每个应用实现 `build`、`update`，并按需要实现 `tick`；页面通过 `smart_band_app_host_t` 注入创建按钮、创建文本、坐标缩放、温度格式化和返回动作等公共能力。

Listing 6.1: 应用分发接口（节选）

```
1 switch (id)
2 {
3     case SMART_BAND_APP_WEATHER:
4         return smart_band_weather_app_build(parent, host);
5     case SMART_BAND_APP_CALCULATOR:
6         return smart_band_calculator_app_build(parent, host);
7     case SMART_BAND_APP_TIMER:
8         return smart_band_timer_app_build(parent, host);
9     case SMART_BAND_APP_MUSIC:
10        return smart_band_music_app_build(parent, host); /* 2048 */
11    case SMART_BAND_APP_MINES:
12        return smart_band_mines_app_build(parent, host);
13    default:
14        return -1;
15 }
```

6.2 八个应用功能

表 6.1: 应用中心功能与实现文件

应用	源文件	可验证功能
Weather	apps/weather_app.c	显示传感器温度、湿度，以及演示用天气范围、风力和天空状态
Calculator	apps/calculator_app.c	数字输入、小数点、退格、清空、四则运算和除零错误

应用	源文件	可验证功能
Timer	apps/timer_app.c	分钟增减、启动/暂停、到零停止和复位
2048	apps/music_app.c	四方向移动、压缩合并、计分、随机新块和结束判断
Stopwatch	apps/stopwatch_app.c	开始/暂停、秒级更新和归零
Mines	apps/mines_app.c	6×6 棋盘、三种难度、相邻雷数和空白区域递归展开
Tetris	apps/tetris_app.c	方块移动、自动下落、碰撞锁定、消行和新局
Wooden Fish	apps/wooden_fish_app.c	点击功德加一、浮动动画、过快提醒、总数和归零反馈

俄罗斯方块使用独立的棋盘状态和定时下落逻辑。图中的页面保留了移动、旋转和重开入口，按钮尺寸按小屏触控区域调整。

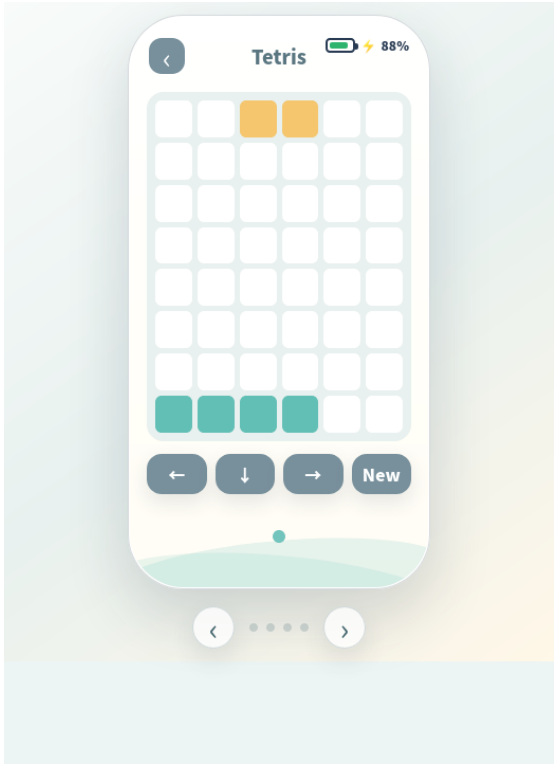


图 6.1: 俄罗斯方块应用页面

6.3 计算器状态机与除零处理

计算器保存当前文本、左操作数、运算符和“下一次输入是否重置”等状态。按钮事件只更新状态，随后统一刷新显示。除数接近零时不继续运算，而是显示 **Err** 并设置错误状态。

Listing 6.2: 计算器除零异常判断

```
1 case '/':  
2     if (rhs > -0.0000001 && rhs < 0.0000001)  
3     {  
4         calc_error();  
5         return false;  
6     }  
7  
8     result = g_lhs / rhs;  
9     break;
```



图 6.2: 计算器的按键与结果显示区域

6.4 2048 的移动与合并

2048 的四个移动方向共用同一套合并规则。实现先根据方向把一行或一列读取为统一顺序的一维数组，再执行“去零压缩、相邻合并、补零”，最后按原方向写回棋盘。

一次移动中每个数字最多合并一次；只有棋盘发生变化时才增加随机方块。

Listing 6.3: 2048 单行压缩与合并核心

```
1 for (int i = 0; i < GAME_SIZE; i++)
2 {
3     if (in[i] != 0)
4     {
5         packed[count++] = in[i];
6     }
7 }
8
9 for (int i = 0; i < count; i++)
10 {
11     if (i + 1 < count && packed[i] == packed[i + 1])
12     {
13         out[out_index] = packed[i] * 2;
14         *score_gain += out[out_index];
15         i++;
16     }
17     else
18     {
19         out[out_index] = packed[i];
20     }
21     out_index++;
22 }
23 }
```

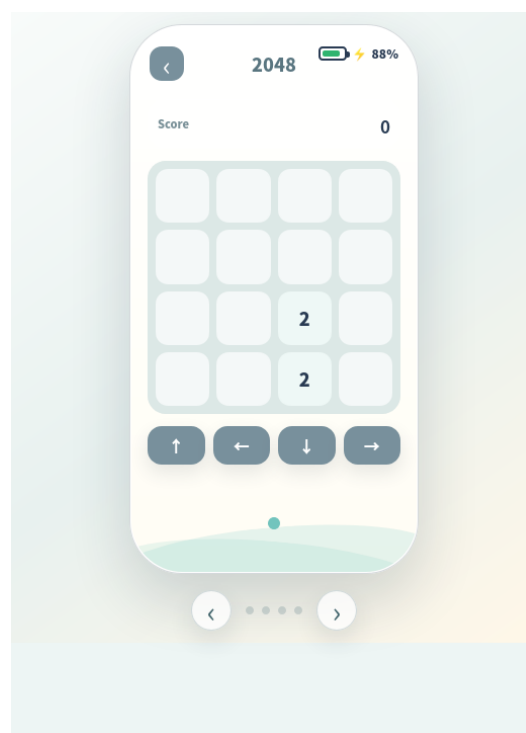


图 6.3: 2048 棋盘、分数与方向控制

6.5 扫雷的递归展开

扫雷使用 64 位位图保存雷区和已揭示单元，6×6 棋盘的状态可以放在少量整数中。安全格被点击后，若周围雷数为零，则递归揭示八邻域；已揭示格、越界格和雷格在入口处直接返回，阻止无限递归。

Listing 6.4: 扫雷空白区域展开判断

```
1 if (index < 0 || index >= SMART_BAND_MINE_CELLS ||  
2     (g_revealed & mine_mask(index)) != 0 ||  
3     (g_mines & mine_mask(index)) != 0)  
4 {  
5     return;  
6 }  
7  
8 g_revealed |= mine_mask(index);  
9 if (mine_neighbor_count(index) != 0)  
10 {  
11     return;  
12 }  
13  
14 /* Recursively reveal valid neighbors. */  
15 mine_reveal_cell(nr * SMART_BAND_MINE_SIZE + nc);
```

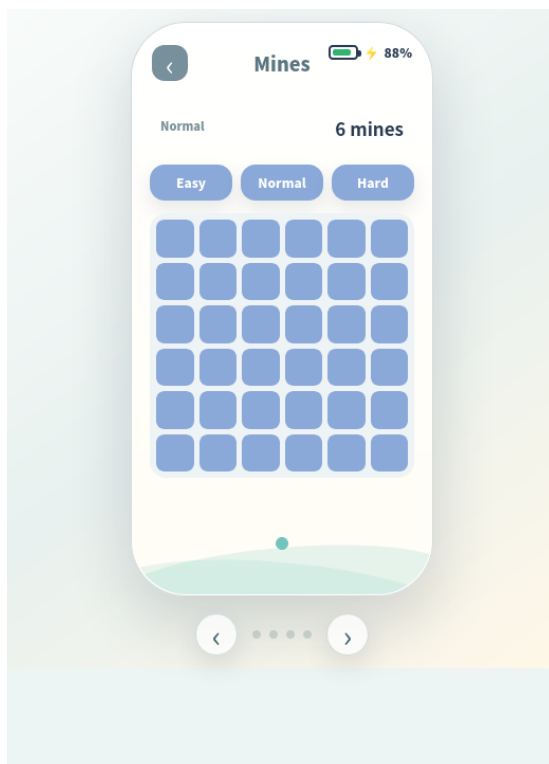


图 6.4: 扫雷棋盘与难度选择区域

6.6 倒计时边界

倒计时运行后不再接受分钟增减；设置值限制在 0–99 分钟。计时归零时自动停止，再次启动会恢复到默认 5 分钟。秒表则维护独立的运行状态和累计秒数，开始、暂停和归零互不混用。



图 6.5: 计时类应用的操作界面

6.7 电子木鱼反馈设计

电子木鱼页面使用卡通主体和木鱼锤作为点击区域。每次敲击后计数加一，同时显示向上移动并逐渐淡出的 Merit+1 文本。连续点击过快时给出节奏提示；归零操作会清空计数并更新提示语。

第七章 编译、运行与一键复现

7.1 环境要求

- Linux 开发环境和可用的 openvela 源码树；
- 对应目标的交叉编译工具链；
- LVGL、LVGL NuttX 适配、libuv UI 循环；
- SENSORS 与 UORB；
- goldfish arm64 模拟器或兼容开发板；
- Python 3（运行模型测试和浏览器静态服务）。

7.2 手动接入源码

假设 openvela 根目录和项目仓库分别为：

/path/to/openvela
/path/to/smart-band-demo

Listing 7.1: 同步原生应用到 openvela packages

```
1 export OPENVELA_ROOT=/path/to/openvela
2 export DEMO_ROOT=/path/to/smart-band-demo
3
4 mkdir -p "$OPENVELA_ROOT/packages/demos/smart_band_basic"
5 rsync -a --delete \
6     "$DEMO_ROOT/openvela_app/smart_band/" \
7     "$OPENVELA_ROOT/packages/demos/smart_band_basic/"
```

若源码树同时使用 apps/packages 镜像目录，需要同步到对应路径。

7.3 配置项

Listing 7.2: goldfish 目标所需配置

```
1 CONFIG_GRAPHICS_LVGL=y
2 CONFIG_LV_USE_NUTTX=y
3 CONFIG_LV_USE_NUTTX_LIBUV=y
4 CONFIG_SENSORS=y
5 CONFIG_UORB=y
6 CONFIG_LVX_USE_DEMO_SMART_BAND_BASIC=y
7 CONFIG_LVX_DEMO_SMART_BAND_BASIC_PRIORITY=100
8 CONFIG_LVX_DEMO_SMART_BAND_BASIC_STACKSIZE=32768
```

也可以通过 menuconfig 检查配置：

```
1 cd "$OPENVELA_ROOT"
2 ./build.sh vendor/openvela/boards/vela/configs/\
3 goldfish-arm64-v8a-ap menuconfig
```

7.4 编译过程

增量构建命令如下：

```
1 cd "$OPENVELA_ROOT"
2 ./build.sh vendor/openvela/boards/vela/configs/\
3 goldfish-arm64-v8a-ap -j2
```

需要排除旧配置影响时执行干净构建：

```
1 ./build.sh vendor/openvela/boards/vela/configs/\
2 goldfish-arm64-v8a-ap distclean -j2
3 ./build.sh vendor/openvela/boards/vela/configs/\
4 goldfish-arm64-v8a-ap -j2
```

编译日志中应出现 `app_lvgl.c`、`sensor_bridge.c`、各应用源文件和 `smart_band_main.c`，最终生成 NuttX 及 vela 镜像产物。

7.5 启动模拟器

模拟器启动时使用智能屏皮肤：

```
1 cd "$OPENVELA_ROOT"
2 ./emulator.sh cmake_out/vela_goldfish-arm64-v8a-ap \
3 -skin xiaomi_smart_screen_10 \
4 -skindir "$OPENVELA_ROOT/prebuilts/emulator/skins/"
```

等待 NuttShell 出现后输入：

```
1 smart_band
```

7.6 Skill 一键复现

仓库提供独立的复现 Skill：

`skills/openvela-smart-band-reproduce/SKILL.md`

复现时先获取 open-vela 官方技能仓库，并使用其中的 quickstart skill 准备环境：

```
1 cd "$OPENVELA_ROOT"
2 git clone https://github.com/open-vela/.claude.git .claude
```

向支持 Skill 的开发助手发出：

帮我搭建 openvela 开发环境

官方环境完成后执行本项目复现脚本：

```
1 bash scripts/reproduce_openvela_demo.sh \
2 --openvela-root /path/to/openvela
```

脚本自动完成以下步骤：

1. 检查或更新官方 `.claude skills`；
2. 将原生应用同步到 `openvela packages` 目录；
3. 在 `goldfish defconfig` 中幂等写入所需配置；
4. 优先执行 CMake 构建，失败后尝试 legacy 构建；
5. 启动静态 HTTP 服务并打开浏览器 Demo；
6. 输出模拟器启动命令和 NSH 中的 `smart_band` 命令。

其中构建回退逻辑如下，能兼容不同阶段的 openvela 源码树：

Listing 7.3: CMake 构建失败后的 legacy 回退

```
1 if ./build.sh "$CONFIG_PATH" --cmake -j"$JOBS"; then
2   log "cmake build completed"
```

```
3     return 0
4 fi
5
6 log "cmake build failed; trying legacy build command"
7 ./build.sh "$CONFIG_PATH" -j"$JOBS" || \
8     die "openvela build failed"
```

第八章 异常处理与可靠性设计

8.1 显示和生命周期错误

程序首先检查 LVGL 是否已初始化，避免同一进程重复建立全局状态。显示初始化失败时输出日志并结束；UI 树创建失败时先释放 NuttX LVGL 资源，再释放 LVGL。本处理防止空显示对象进入 libuv 循环，也避免失败路径遗漏资源清理。

Listing 8.1: 显示与 UI 初始化失败处理

```
1 if (result.disp == NULL)
2 {
3     printf("smart_band: LVGL display initialization failed\n");
4     lv_deinit();
5     return 1;
6 }
7
8 if (smart_band_lvgl_create(NULL) != 0)
9 {
10    printf("smart_band: failed to create LVGL UI\n");
11    lv_nuttx_deinit(&result);
12    lv_deinit();
13    return 1;
14 }
```

8.2 传感器异常降级

- bridge 初始化时所有文件描述符先设为 -1，读取和关闭前均检查有效性；
- 单个传感器打开失败不会终止应用，其他数据源继续工作；
- 采样间隔 ioctl 返回不支持时只记录日志，不视为致命错误；
- 非阻塞读取暂时没有样本时保留最后一次有效值或模型值；
- step counter 不可用时切换到加速度推导；
- ambient temperature 不可用时尝试普通 temperature 节点；
- 电池 ioctl 失败时不更新本轮电池状态，不影响健康和应用页面。

8.3 输入和数值边界

表 8.1: 主要边界保护

对象	保护方式
心率	只接收大于零的样本，并限制为合理的显示整数范围
步数	限制在 0-99999；目标限制为 1000-50000
温湿度	温度保留传感器原始整数意义；湿度限制为 0-100%
电池	容量限制为 0-100%，充电状态单独判断
页面	模运算循环，空状态指针立即返回
计算器	文本长度受缓冲区限制，除零进入 Err 状态
倒计时	0-99 分钟，运行中禁止修改，到零自动停止
游戏	棋盘索引、移动范围、递归入口和结束状态均检查

8.4 复现脚本错误处理

复现脚本启用未定义变量检查,对未知参数、缺失参数、源码目录不存在、defconfig 缺失和构建失败使用明确错误信息退出。若 openvela 尚未安装，脚本以专用状态码结束，并打印交给官方 quickstart skill 的下一条提示，而不是在未知环境中继续修改。

第九章 测试与验证

9.1 模型自动测试

`tests/test_watch_model.py` 对无需硬件的稳定规则进行测试，包括：

- 时间和日期格式；
- 页面正向、反向循环；
- 600 秒模拟数据范围；
- 步数进度计算；
- 目标步数上下界。

执行命令：

```
1 python3 tests/test_watch_model.py
```

9.2 传感器联动测试

模拟器控制台需要认证令牌。仓库脚本通过 5554 端口依次写入心率、温度、湿度、加速度和电池容量，并读取状态用于核对：

```
1 SMART_BAND_ROLL_LOOPS=4 SMART_BAND_ROLL_DELAY=2 \  
2 scripts/roll_emulator_sensors.sh
```

预期结果为心率页数值随 `heart-rate` 变化、天气温湿度随传感器变化、步数在加速度变化时增长、电池百分比及填充宽度变化，充电时出现闪电标识。

9.3 人工验收流程

1. 启动 `smart_band`，确认表盘时间、日期、电池和摘要卡片可见；
2. 连续左滑四次，确认表盘、心率、计步、应用中心形成闭环；
3. 从应用中心继续后翻，确认能够回到表盘；

4. 在计步页调节目标，检查进度和剩余步数同步变化；
5. 进入八个应用，验证按钮、返回和状态更新；
6. 在 2048 中测试四方向移动，在扫雷中切换难度，在倒计时中修改时间；
7. 调节模拟器传感器，比较控制台值与 UI 显示；
8. 关闭某个可选设备节点或不推送样本，确认页面保持运行而非崩溃。

9.4 验证边界说明

Python 测试验证模型规则，浏览器 Demo 验证视觉与基础交互，goldfish 模拟器验证 openvela 编译、LVGL 运行和传感器链路。三者用途不同，不能用浏览器模拟数据替代原生传感器验收。

第十章 开源合规与可维护性

10.1 开源文件

仓库根目录提供：

- LICENSE：项目许可证正文；
- NOTICE：项目归属、用途和上游关系说明；
- THIRD_PARTY_NOTICES.md：第三方依赖和资源声明；
- README.md：项目概览、构建、运行和异常说明；
- docs/openvela-runbook.md：详细复现手册。

本仓库只提交智能手环应用、演示和复现辅助内容。openvela SDK、NuttX、LVGL、libuv、模拟器和工具链属于各自上游项目，应遵循对应许可证。本项目没有把上游完整系统代码重新声明为自身成果。

10.2 扩展新应用

新增应用时建议遵循以下步骤：

1. 在 apps/ 下创建独立 C 文件；
2. 在 smart_band_apps.h 增加应用 ID；
3. 在 smart_band_apps.c 注册名称、颜色和 build/update/tick 分发；
4. 在 icon_assets.h/.c 增加图标资源；
5. 在 CMakeLists 和 Makefile 中同时加入源文件；
6. 在模拟器中检查小屏排版、按钮触摸范围、返回和状态边界。

第十一章 总结与展望

目前，项目已经能够在 `openvela/NuttX` 环境中完成编译、启动和基本交互验证。表盘、心率、计步和应用中心四个主页面均已实现，天气、计算器、倒计时、2048、秒表、扫雷、俄罗斯方块和电子木鱼八个小应用也分别具备可操作的功能入口。传感器部分已形成较完整的数据链路：`goldfish` 模拟器或 `uORB` 设备节点产生样本后，由桥接层统一完成设备打开、非阻塞读取、数据转换和状态更新，再写入 `smart_band_state_t`，最终由 `LVGL` 页面按固定周期刷新显示。心率、步数、加速度、温度、湿度、电池容量和充电状态均有对应的处理路径，页面代码不直接管理设备文件描述符，从而降低了界面层与设备层之间的耦合。

在稳定性方面，当前实现已经考虑了模拟器和嵌入式环境中常见的异常情况。设备节点不存在时，应用不会直接退出；传感器暂时没有新样本时，页面继续保留上一次有效值或模型默认值，避免数据显示为零、空白或频繁跳动；独立计步器不可用时，可以根据加速度变化估算步数；环境温度节点打开失败时，会继续尝试普通温度节点；电池读取失败时只跳过本轮状态更新，不影响心率、计步和其他页面运行。项目还对页面编号、步数目标、进度比例、倒计时范围、计算器输入、棋盘索引和传感器数值进行了边界检查，并在显示初始化或 UI 创建失败时释放已申请的资源，减少异常路径中的崩溃和资源泄漏风险。

从工程组织看，项目已经将程序入口、状态模型、传感器桥接、主页面和小应用分别拆分到独立模块中。`smart_band_main.c` 负责 `LVGL`、`NuttX` 显示适配和 `libuv` 事件循环，`watch_model.c` 负责时间和公共状态，`sensor_bridge.c` 负责设备访问，`app_lvgl.c` 负责主页面交互，八个小应用则分别位于 `apps/` 目录。项目同时保留 `CMake` 和 `NuttX legacy Make` 两种构建入口，并提供模型自动测试、模拟器传感器联动脚本、运行手册和一键复现脚本。相关构建配置、开源许可证、第三方声明和复现说明也随仓库一并提供，使评审人员或后续开发者能够按照统一步骤重新构建、运行和验证项目。

目前的验证重点仍然集中在 `goldfish` 模拟器环境。后续工作首先应转向真实硬件测试，重点检查不同开发板上的显示驱动、触摸响应、传感器节点名称、电池接口和屏幕尺寸差异，并进一步测量程序运行时的内存占用、CPU 占用、传感器采样频率和整体功耗。对于计步功能，可以在真实运动数据基础上引入滤波、峰值检测、姿态变化识别和误触发抑制，提高步数统计的准确性；对于心率、睡眠和压力等健康指标，也可以结合更稳定的采样窗口和数据质量判断，减少单次异常样本对页面结果的影响。

在功能扩展方面，后续可以增加本地数据持久化，使步数、运动目标、应用状态和历史健康数据在重启后继续保留；增加通知中心、闹钟、日程提醒和蓝牙同步能力，

使手环能够与手机或其他设备交换数据；增加配置页面，让用户调整步数目标、时间格式、传感器采样周期和界面主题。中文界面方面，需要为 LVGL 接入经过裁剪的中文字库，并重新检查不同字号下的文本宽度、换行和按钮布局，避免因字体资源过大影响存储空间和运行内存。

现有模块边界已经将页面显示、状态管理、设备读取和小应用逻辑分开，因此后续新增功能可以按模块逐步接入。新增传感器时，可以在桥接层增加设备路径和字段转换；新增小应用时，可以在 `apps/` 目录实现独立文件并注册到应用中心；适配新开发板时，主要调整显示、触摸和设备节点配置，而不需要重写整个界面框架。整体来看，当前版本已经完成从前端原型到 openvela 原生应用的基本工程化过程，并为真实硬件适配、健康算法优化和设备互联功能提供了可继续扩展的基础。

附录 A 快速复现命令清单

```
1 # 1. Prepare official openvela skills
2 cd /path/to/openvela
3 git clone https://github.com/open-vela/.claude.git .claude
4
5 # 2. Ask the official skill to prepare openvela
6 # Prompt: 帮我搭建 openvela 开发环境
7
8 # 3. Integrate and build this project
9 cd /path/to/smart-band-demo
10 bash scripts/reproduce_openvela_demo.sh \
11     --openvela-root /path/to/openvela
12
13 # 4. Start simulator
14 cd /path/to/openvela
15 ./emulator.sh cmake_out/vela_goldfish-arm64-v8a-ap \
16     -skin xiaomi_smart_screen_10 \
17     -skindir /path/to/openvela/prebuilts/emulator/skins/
18
19 # 5. Start app in NuttShell
20 smart_band
21
22 # 6. Roll simulator sensor values in another terminal
23 cd /path/to/smart-band-demo
24 SMART_BAND_ROLL_LOOPS=4 SMART_BAND_ROLL_DELAY=2 \
25     scripts/roll_emulator_sensors.sh
26
27 # 7. Run model tests
28 python3 tests/test_watch_model.py
```